

Devcpp

Dev-C++

介绍

Dev-C++ 是一套用于开发 C/C++ 程序的自由的集成开发环境（IDE），并以 GPL 作为分发许可，使用 MinGW 及 GDB 作为编译系统与调试系统。Dev-C++ 运行在 Microsoft Windows 下。

Dev-C++ 的优点在于界面简洁友好，安装便捷，支持单文件编译，因此成为了许多入门 OI 选手以及 C++ 语言初学者的首选。在 NOIP 中，提供 Windows 作为比赛系统的省份一般预置 Dev-C++。

Dev-C++ 起源于 Colin Laplace 编写的 Bloodshed Dev-C++。该版本自 2005 年 2 月 22 日停止更新。2006 年，Dev-C++ 主要开发者 Colin Laplace 曾经对此作出了解释：「因忙于现实生活的事务，没有时间继续 Dev-C++ 的开发。」

Orwell Dev-C++ 是 Dev-C++ 的一个衍生版本，由独立程序员 Orwell (Johan Mes) 开发并维护。其对原版 Dev-C++ 进行了错误修正，并更新了编译器版本。一般而言，Dev-C++ 5.x 均为 Orwell Dev-C++。其最后一次更新于 2015 年，版本为 5.11。

Embarcadero Dev-C++¹是 Bloodshed Dev-C++ 和 Orwell Dev-C++ 的继任者。2020 年，Embarcadero 赞助并接手了原有的 Dev-C++ 项目，继续开发。Embarcadero Dev-C++ 加入了对高 DPI 的支持，更新了编译器以加入更新版本的 C++ 标准支持，以及暗色模式。

以上的 Dev-C++ 分发都被认为是「官方的」。此外，在 2015 年 Orwell Dev-C++ 停止更新后，因为教学需要，一位来自中国的个人开发者 [royqh1979](#) 决定继续开发他的 Dev-C++ 个人分支，命名为小熊猫 Dev-C++²，集成了智能提示和高版本的 MinGW64，非常便于国内的个人使用和学习。

小熊猫 Dev-C++ 6.7.5 版本发布后，作者使用 qt5 开发了全新的小熊猫 C++³，可在 windows、linux 和 macos 等系统下原生运行。小熊猫 C++ 的界面与 Dev-C++ 相似，除了提供和 Dev-C++ 相似但更加完善的单文件编译、调试、语法高亮、搜索/替换等功能外，还提供了诸如 **暗色主题**、**代码智能提示**、**变量/函数重命名**、**切换/自动识别文件编码** 等现代 IDE 常见的基本功能。此外小熊猫 C++ 还具备与 CP Editor 类似的试题集功能，可以自行编写或从常见的 **OJ 竞赛** 网站上下载试题样例，自动运行和测试程序。

使用教程

常用快捷键

文件部分

- Ctrl + N: 创建源代码
- Ctrl + O: 打开文件
- Ctrl + W: 关闭文件
- Ctrl + Shift + W: 关闭所有文件
- Ctrl + P: 打印文件

格式部分

- Ctrl + /: 注释和取消注释
- Tab: 缩进
- Shift + Tab: 取消缩进

行操作

- Ctrl + E: 复制行
- Ctrl + D: 删除行
- Ctrl + Shift + Up: 向上移动
- Ctrl + Shift + Down: 向下移动

跳转部分

- Ctrl + F: 搜索
- Ctrl + R: 替换
- F3: 搜索下一个
- Shift + F3: 搜索上一个
- Ctrl + G: 到指定行号
- Shift + Ctrl + G: 到指定函数
- Ctrl + [1 ~ 9]: 设置书签
- Alt + [1 ~ 9]: 跳转书签

显示部分

- Ctrl + 滚轮: 字号放大或缩小
- Ctrl + F11: 全屏或恢复

运行部分

- F9: 只编译
- F10: 只运行
- F11: 编译并运行
- F12: 全部重新编译

调试部分

- F2: 转到断点
- F4: 设置断点或取消
- F5: 调试运行
- F6: 停止
- F7: 逐步调试

调试流程

1. 将编译器配置设定为 TDM-GCC 4.9.2 64-bit Debug
2. 按 F4 设置或取消调试断点
3. 将光标放置在变量上, 按 Alt + A 向调试窗口添加监控变量
4. 按 F5 启动调试
5. 按 F7 或 Alt + N 逐步调试
6. 按 Alt + S 跳至下一个调试断点
7. 按 F6 停止调试

扩展

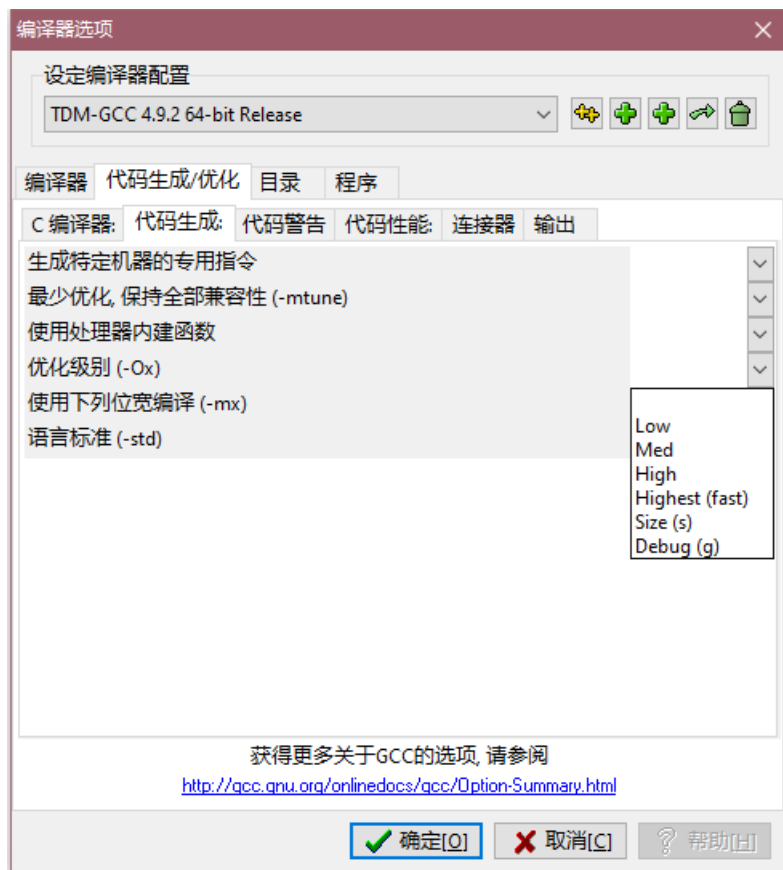
增加编译选项

点击工具 -> 编译选项, 然后选择 "代码生成/优化" 选项卡, 下面介绍笔者常用的几个编译选项。

开启优化

优化代码运行时间或占用空间。

选择 "代码生成" 子选项卡中的 "优化级别（-Ox）" 选项标签。



更换语言标准

使用新语言特性或试图让代码在旧标准下编译。

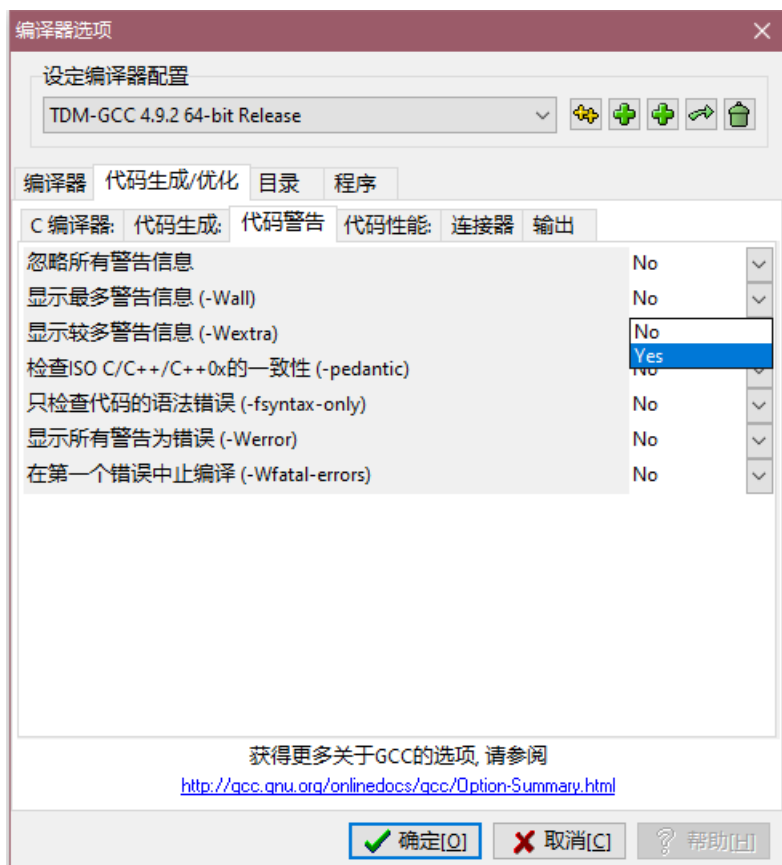
选择 "代码生成" 子选项卡中的 "语言标准（-std）" 选项标签。



显示最多警告信息

查错小助手。

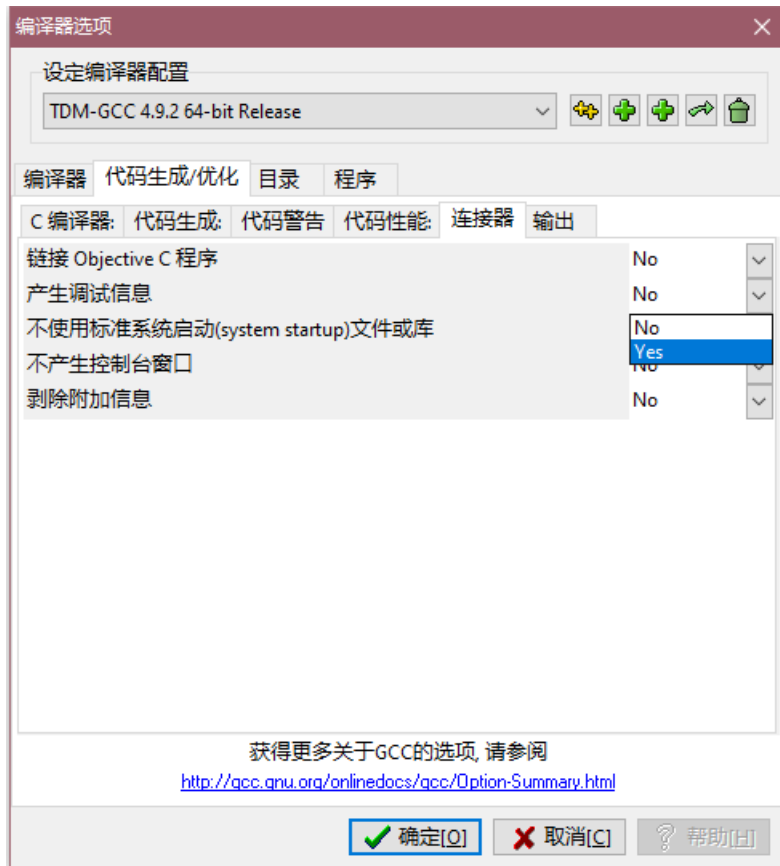
选择 "代码警告" 子选项卡中的 "显示最多警告信息 (-Wall)" 选项标签。



生成调试信息

当显示 "项目没有调试信息，您想打开项目调试选项并重新生成吗？" 点击后闪退或想使用调试功能时需开启此功能。

选择 "连接器" 子选项卡中的 "产生调试信息" 选项标签。



编译小 trick

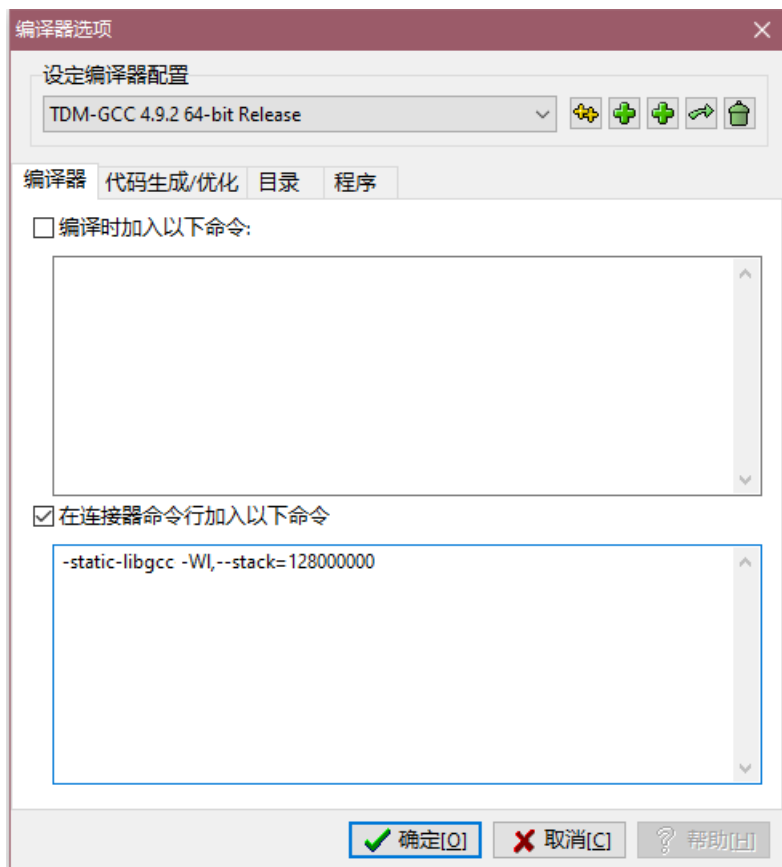
点击工具 -> 编译选项，然后选择 "编译器" 选项卡，接下来介绍几个常用 trick。

开大栈

防止 DFS 爆系统栈之类的情況出现。

在 "连接器命令行加入以下命令" 中加入 `-Wl,--stack=128000000` 命令。

此命令将栈开到了约 128MB 的大小，有需要可以自行增加。



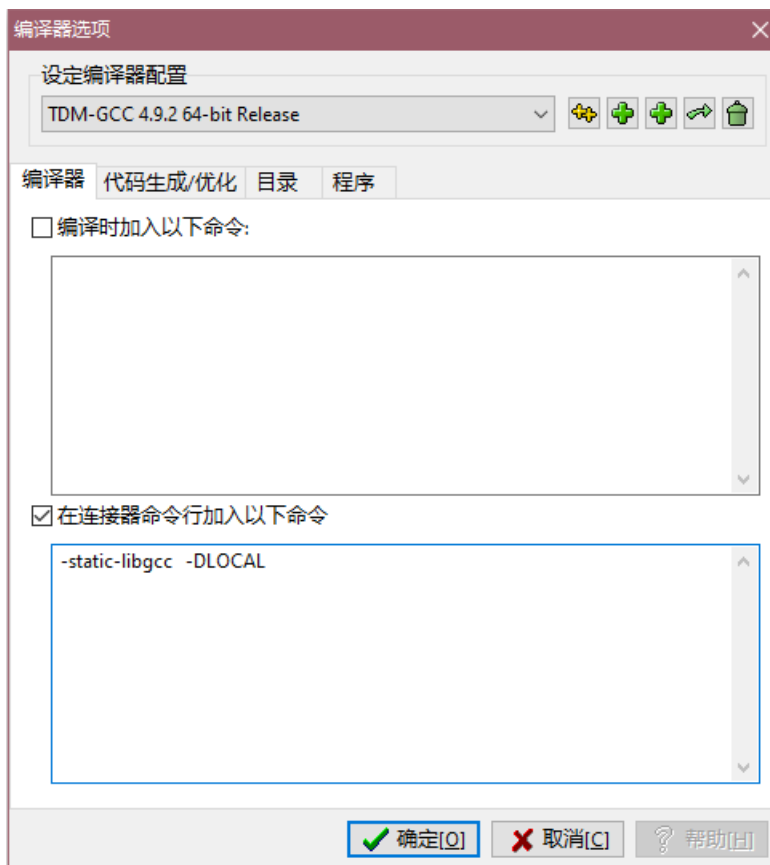
定义宏

方便本地评测使用文件输入输出或作其他用途。

在 "连接器命令行加入以下命令" 中加入 `-D[String]` 命令。

其中 `[String]` 改为你需要的宏名。

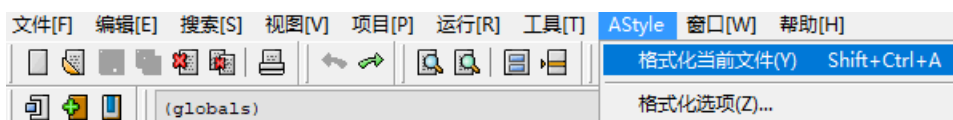
如图，当开启编译选项后便可将以下代码从 `test.in` 文件读入数据并在 `test.out` 文件中输出。



```
1 #ifdef LOCAL
2 freopen("test.in", "r", stdin);
3 freopen("test.out", "w", stdout);
4 #endif
```

代码格式化

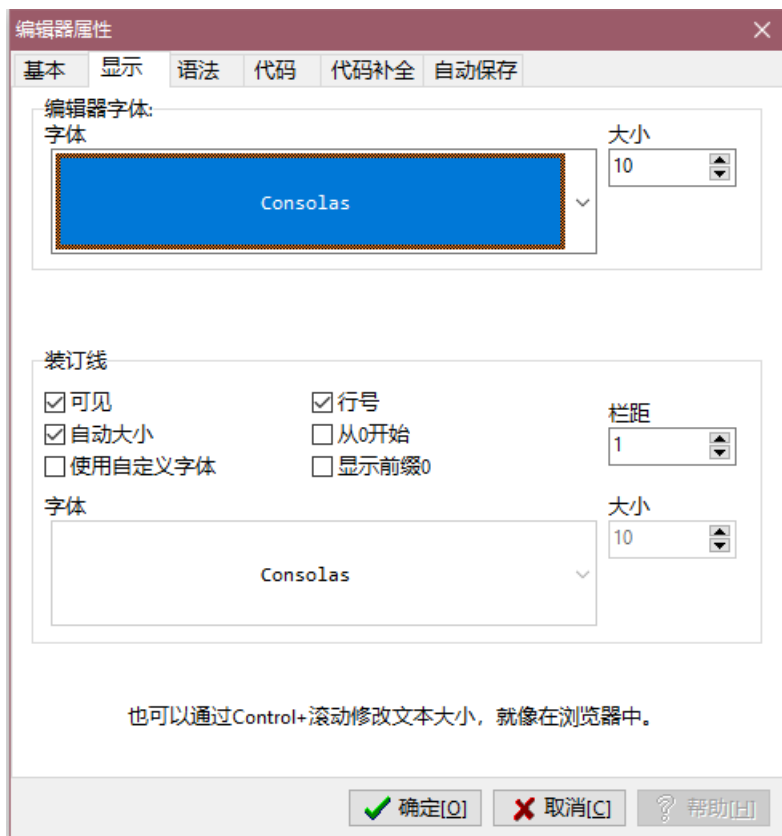
点击 Astyle-> 格式化当前文件 或 按 Ctrl+Shift+A 进行代码格式化。



美化

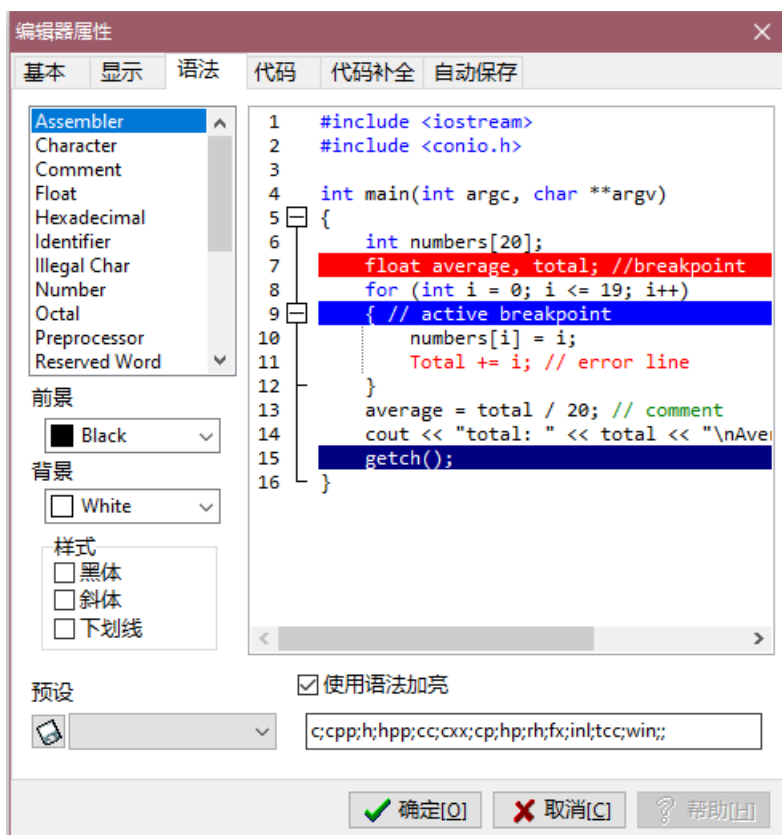
字体

点击工具 -> 编辑器选项，然后选择 "显示" 选项卡。



主题

点击工具 -> 编辑器选项，然后选择 "语法" 选项卡，可以使用预设主题，也可以自行调整。



参考资料

1. 项目源代码托管于 [GitHub](#) 和 [SourceForge](#). ↩

2. 源代码托管于 [Github ↗](#)
 3. 项目官网位于 [小熊猫 C++](#), 源代码托管于 [Github ↗](#)
-

本页面最近更新: 2024/6/1 11:14:01, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [CamberLoid](#), [DawnMagnet](#), [Doveqise](#), [hsfzLZH1](#), [ksyx](#), [ouuan](#), [royqh1979](#), [sshwy](#), [topdevst](#), [wangqingshiyu](#), [CCXXXI](#), [CoelacanthusHex](#), [Early0v0](#), [Enter-tainer](#), [Henry-ZHR](#), [Ir1d](#), [Konano](#), [maye76](#), [NachtgeistW](#), [NanoApe](#), [StudyingFather](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用